

Introduction

Lors de ce projet Java, nous avons codé la génération d'un labyrinthe, avec le choix d'un labyrinthe parfait ou non, la modification des murs, l'historique de ces changements, ainsi que la possibilité de sauvegarder un labyrinthe et de le restaurer ensuite. Pour résoudre ces labyrinthes, nous avons choisi trois algorithmes :

- **Trémaux** : À chaque intersection, il choisit un chemin non exploré, et lorsqu'il rencontre un cul-de-sac, il revient en arrière en suivant le parcours précédent jusqu'à trouver une nouvelle route, jusqu'à déterminer le ou les chemins possibles.
- **"Dead End filling"** : cet algorithme détecte les culs-de-sac et remonte le chemin jusqu'au croisement le plus proche, évitant ainsi les zones sans issue. Il est réalisable uniquement informatiquement et ne peut pas être utilisé pour un vrai labyrinthe.
- **Dijkstra** : il s'agit de la recherche du plus court chemin classique.

Organisation de l'équipe

Nous avons travaillé en groupe, surtout en appel, en se partageant souvent nos écrans pour suivre l'avancée du code et discuter des idées de chacun. Cela nous a permis d'échanger facilement, de voir où nous en étions et de régler les problèmes au fur et à mesure. Si une personne avançait seule, elle partageait ses avancées au groupe, en rapportant les problèmes rencontrés et les idées de modifications.

Problèmes et solutions

Nous avons rencontré plusieurs problèmes pendant ce projet :

- **Affichage** : Lors de la génération du labyrinthe, si la taille était trop importante, les boutons et le labyrinthe se superposaient. Nous avons donc imposé une taille maximale et déplacé les boutons sur le côté afin d'éviter ce problème et de clarifier l'affichage.
- **GridPane** : Pour la modification d'une case, nous utilisons une GridPane. Cependant, il arrivait que la sélection d'une case ne soit pas correcte : au lieu de choisir la case visée, une case voisine était sélectionnée, notamment lorsque le labyrinthe était trop petit (<400 unités d'aire) . Nous avons donc imposé une taille de 30 aux cases lorsque c'est le cas, sinon elles prennent la valeur par défaut d'une cellule de GridPane. Nous avons demandé de l'aide à Copilot de GitHub afin de régler ce problème.
- **JavaFX** : Nous avons rencontré de nombreux problèmes avec l'interface graphique JavaFX, ne fonctionnant pas correctement sur nos ordinateurs. Nous avons donc utilisé une version antérieure de java (java 8) qui contient initialement la bibliothèque JavaFX. On peut aussi utiliser une version plus récente et télécharger la bibliothèque JavaFX associée, mais cela nous a posé problème.
- **Bug restauration** : Lors de la mise en place de la restauration, un problème est apparu : après avoir restauré un labyrinthe sauvegardé, la résolution pas-à-pas ne fonctionnait plus. Ce problème a été résolu, il était seulement dû à un problème de variable.

- **Dead End filling** : Lors du passage au labyrinthe imparfait, l'algorithme n'était plus adapté, notamment pour les cycles. Nous avons donc dû modifier le code afin qu'il reconnaisse ces cycles présents dans des zones fermées et de les enlever du chemin.

Limites fonctionnelles

Afin de simplifier l'affichage et la génération (ainsi que pour le GridPane), nous avons imposé une taille limite : 20*20 pour la version JavaFX et 30*30 pour la version terminal. Toutes valeurs supérieures seront ramenées à la valeur limite.

La génération étant complexe, nous avons de plus imposé la position de l'entrée et de la sortie, avec l'entrée en haut à gauche, et la sortie en bas à droite.

Pour éviter une génération excessive de tableaux, nous avons de plus enlevé l'option de générer le tableau pas-à-pas pour la version terminal, ainsi que pour la résolution des trois algorithmes. Il sera évidemment problématique de devoir générer un tableau à chaque case parcourue en version terminal.

Planning de réalisation

Semaine 1-2 : Nous avons débuté en créant les diagrammes d'utilisation et de classe pour structurer le projet. Dans un premier temps, nous avons implémenté la génération d'un labyrinthe parfait ainsi que son affichage de base à l'aide de l'interface graphique JavaFX. Par la suite, nous avons codé l'algorithme de résolution de Trémaux. Afin de rendre la résolution plus visuelle, un version pas-à-pas a été ajoutée permettant d'afficher l'état de la progression de l'algorithme à chaque étape, tout en ajoutant les statistiques (temps d'exécution, cases parcourues, chemin le plus court).

Semaine 3 : Nous avons tout d'abord ajouté une fonctionnalité permettant de modifier les murs des cases, afin d'offrir plus de contrôle à l'utilisateur sur le labyrinthe. Un bouton Retour a également été intégré, permettant à l'utilisateur de revenir facilement au menu, évitant de relancer l'intégralité du programme. Concernant la génération du labyrinthe, l'utilisateur peut choisir la vitesse de création. Par ailleurs, nous avons implémenté l'algorithme de Dijkstra pour proposer une méthode de résolution alternative, ainsi que l'algorithme "Dead End filling", tout en intégrant leur version pas-à-pas. Enfin, plusieurs bugs ont été corrigés sur les deux nouveaux algorithmes, mais aussi sur l'affichage notamment ceux liés à la GridPane.

Semaine 4 : Pour finaliser notre projet, nous avons introduit une génération de labyrinthe non parfaite en combinant la génération d'un labyrinthe parfait et certaines modifications. Cette génération est accompagnée d'une version pas-à-pas, offrant la possibilité de visualiser l'évolution du processus de création du labyrinthe en temps réel. De plus, nous avons ajouté un système d'historique et de restauration, permettant à l'utilisateur de voir les modifications apportées au labyrinthe et de revenir au labyrinthe sauvegardé. Nous avons corrigé plusieurs bugs liés notamment à l'affichage et au Dead End filling pour les labyrinthes imparfaits afin de garantir la résolution pour un labyrinthe quelconque.

Diagramme d'utilisation

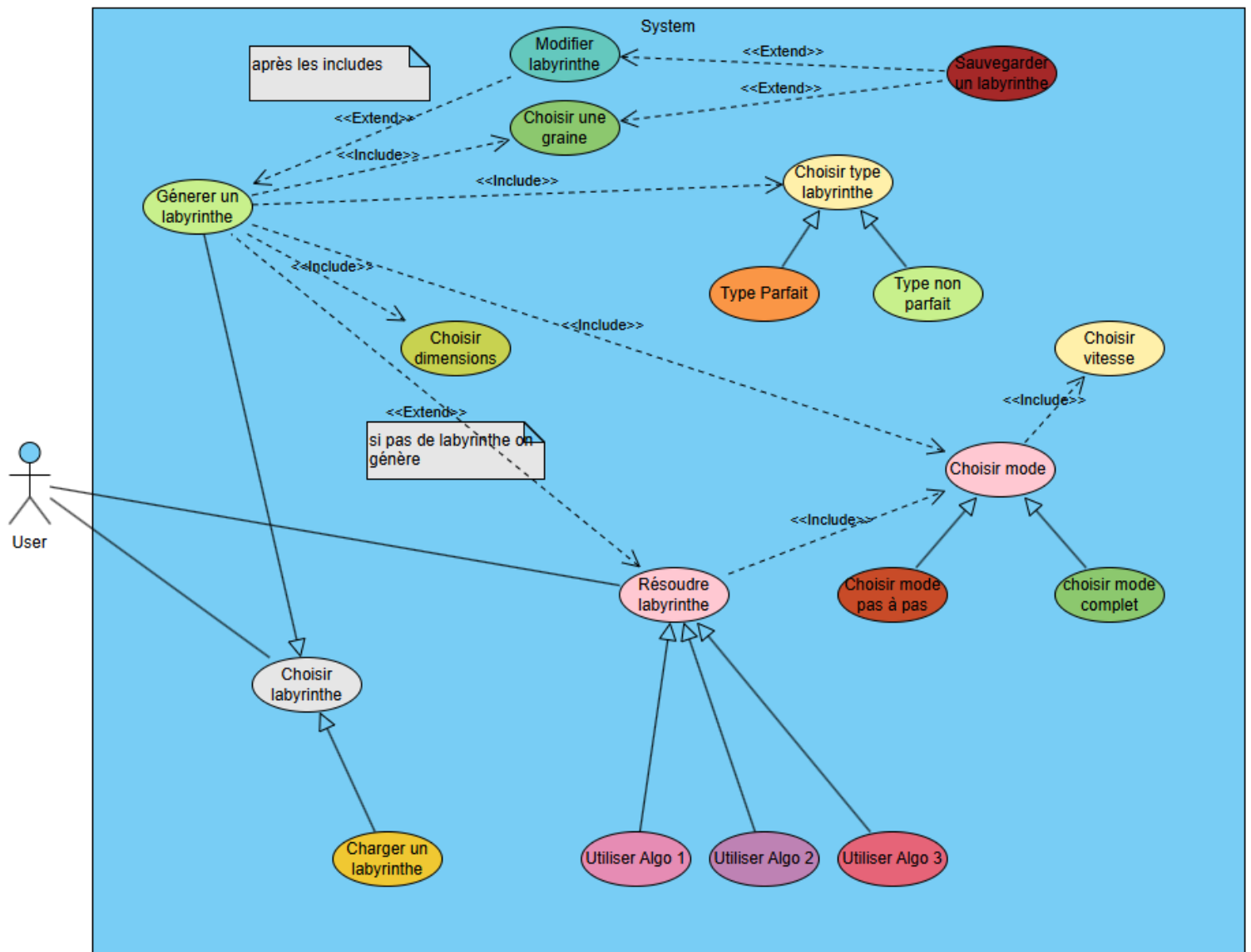


Diagramme de classe

(https://app.diagrams.net/?splash=0#G11fTqiBcgCh_a0zow32CtQnrQ25oX-65H#%7B%22pageId%22%3A%22EnLaLpWhG9iAXsXka8f%22%7D)

